

Validation de syllogismes complexes à l'aide des diagrammes de Venn.

1 Avant-propos

L'évaluation de TP, c'est-à-dire ce projet, est un examen universitaire comptant pour la délivrance du diplôme national de licence.

À ce titre, toute fraude ou plagiat entraîne la rédaction d'un procès-verbal à l'attention de la section disciplinaire de l'université, qui peut infliger, en plus de la nullité de l'épreuve (note de 0), une sanction allant de l'avertissement à l'exclusion définitive de tout établissement public d'enseignement supérieur.

Ainsi, si vous utilisez un morceau de code que vous n'avez pas produit, vous devrez indiquer très clairement sa source (site internet, camarade de promotion, intelligence artificielle, etc.) : vous ne serez pas accusé de plagiat, mais ce morceau de code ne sera pas compté dans votre note finale.

2 Modalités de réalisation et de rendu

Le projet est à réaliser par groupes de **deux étudiants maximum**. Vous devrez rédiger un rapport de projet contenant :

1. les réponses aux questions contenues dans l'énoncé du projet,
2. les choix de programmation (structures de données, fonctions récursives terminales ou non, etc.),
3. les fonctions utilitaires que vous avez ajoutées (en-tête et documentation, pas le code),
4. les difficultés rencontrées et les solutions apportées.

Vous devrez rendre le projet (rapport et code source dans une archive) pour le dimanche 21 décembre 2025 au plus tard, en déposant celui-ci sur universitice dans le dépôt correspondant. Une soutenance de 15 minutes par binôme aura lieu le mardi 6 ou le vendredi 9 janvier 2026 afin de vous interroger sur votre réalisation : vous serez interrogés au tableau sur l'intégralité de votre code et sur l'algorithme implanté, et vous devrez répondre à des questions sur ceux-ci.

3 Introduction

Nous avons étudié ce semestre les syllogismes et leur conversion en diagramme de Venn. Ces syllogismes utilisent des cas particuliers des formules du calcul des prédicats, où les prédicats sont unaires, quantifiés une et une seule fois et portant sur une unique variable. De plus, les syllogismes étudiés jusqu'à présent ne comportaient qu'une unique conclusion.

Dans ce projet, nous allons étendre cette étude à des syllogismes dont les prémisses et la conclusion peut être une combinaison Booléenne (c'est-à-dire des combinaisons à base de conjonctions, disjonctions et négations) de formules quantifiées. De tels syllogismes seront appelés *syllogismes complexes*.

Le but de ce projet est d'étudier un algorithme afin de déterminer la validité de tels syllogismes à l'aide des diagrammes de Venn.

4 L'insuffisance de la notion de compatibilité : un exemple simple

Jusqu'à présent, la validité des syllogismes était déterminée à l'aide de la notion de compatibilité entre les diagrammes de Venn des prémisses et ceux de la conclusion : pour qu'un syllogisme soit valide, chaque diagramme construit à partir des prémisses devait être compatible avec au moins un diagramme construit à partir de la conclusion.

Cependant, cette notion de compatibilité n'est pas suffisante pour déterminer la validité de syllogismes complexes. En effet, des syllogismes complexes peuvent être valides, bien que certains diagrammes construits à partir des prémisses ne soient compatibles avec aucun diagramme construit à partir de la conclusion étendue.

Considérons par exemple les formules suivantes :

$$p_1 = \forall x (b(x) \rightarrow a(x)),$$

$$p_2 = \forall x (a(x) \rightarrow b(x)),$$

$$p_3 = \exists x (a(x) \wedge \neg b(x)),$$

et considérons le syllogisme à conclusion étendue avec pour prémisses p_1 et pour conclusion étendue $p_2 \vee p_1 \wedge p_3$, qui sera alors noté pour plus de concision sous la forme suivante :

$$p_1 \stackrel{?}{\vdash} p_2 \vee p_1 \wedge p_3.$$

La prémisses p_1 admet un unique diagramme de Venn, noté D et représenté ci-dessous :

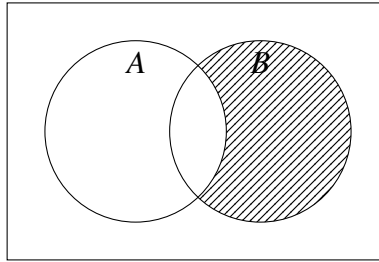


FIGURE 1 – Le diagramme D de p_1 .

Pour calculer les diagrammes de la conclusion, il suffit de calculer les listes respectives des diagrammes de p_1 , p_2 et p_3 séparément, puis de combiner celles-ci en considérant les opérations suivantes¹, pour deux listes $\ell = [d_1, \dots, d_n]$ et $\ell' = [d'_1, \dots, d'_m]$ de diagrammes :

- la **disjonction** ℓ et ℓ' est la **concaténation** des deux listes de diagrammes :

$$\ell \vee \ell' = [d_1, \dots, d_n, d'_1, \dots, d'_m],$$

- la **conjonction** de deux listes de diagrammes est la **combinaison cartésienne** des conjonctions des diagrammes pour lesquelles la conjonction est définie² :

$$\ell \wedge \ell' = [d_i \wedge d'_j \mid 1 \leq i \leq n, 1 \leq j \leq m, d_i \wedge d'_j \text{ n'est pas None}].$$

On pourra alors vérifier que la conclusion étendue admet les deux diagrammes D'_1 et D'_2 suivants :

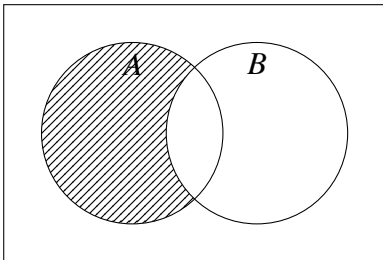


FIGURE 2 – Le diagramme D'_1 .

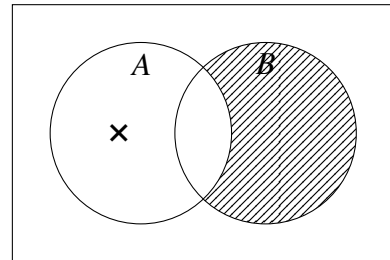


FIGURE 3 – Le diagramme D'_2 .

1. À rapprocher des opérations ensemblistes de la mise en forme clausale **disjonctive**, forme duale de la forme clausale conjonctive.
2. Pour rappel, la conjonction de diagrammes est une fonction *partielle*, c'est-à-dire pouvant renvoyer None en OCaml.

Notons que le diagramme D de la prémisses n'est compatible avec aucun des diagrammes D'_1 et D'_2 de la conclusion. Pourtant, le syllogisme $p_1 \vdash^? p_2 \vee p_1 \wedge p_3$ est valide. En effet, pour toute situation dans laquelle p_1 est vérifiée, on peut supposer qu'une des deux situations suivantes est vraie³ :

- soit la zone $A \cap \neg B$ du diagramme est vide,
- soit la zone $A \cap \neg B$ du diagramme est non-vide⁴.

En considérant ces deux choix, on sait alors que pour toute situation compatible avec le diagramme D de la prémisses, soit la zone $A \cap \neg B$ du diagramme est vide et il y a alors compatibilité avec le diagramme D'_1 de la conclusion, soit la zone $A \cap \neg B$ du diagramme est non-vide et il y a alors compatibilité avec le diagramme D'_2 de la conclusion. Ainsi, le syllogisme $p_1 \vdash^? p_2 \vee p_1 \wedge p_3$ est valide, bien que le diagramme D de la prémisses ne soit compatible avec aucun des diagrammes D'_1 et D'_2 de la conclusion.

5 Une notion plus complexe : l'existence de diagrammes contradictoires

Si le syllogisme de l'exemple précédent est valide, c'est que quelle que soit la valeur choisie pour la zone $A \cap \neg B$ afin d'étendre le diagramme de la prémisses, l'extension (c'est-à-dire le diagramme obtenu) est compatible avec au moins un des diagrammes de la conclusion étendue. À l'extrême, cela veut dire que tout diagramme complet (c'est-à-dire avec une contrainte de vacuité ou de non-vacuité sur chaque zone) obtenu en étendant le diagramme de la prémisses est compatible avec au moins un des diagrammes de la conclusion étendue.

À l'inverse, pour que le syllogisme soit invalide, il aurait fallu trouver une situation dans laquelle un diagramme des prémisses peut être étendu en un diagramme complet qui n'est compatible avec aucun des diagrammes de la conclusion étendue.

Par exemple, considérons les formules suivantes :

$$p_4 = \forall x (a(x) \vee b(x)),$$

$$p_5 = \forall x (a(x)),$$

$$p_6 = \forall x (b(x)),$$

et le syllogisme complexe suivant :

$$p_4 \vdash^? p_5 \vee p_6.$$

Le diagramme de la prémisses p_4 est le suivant :

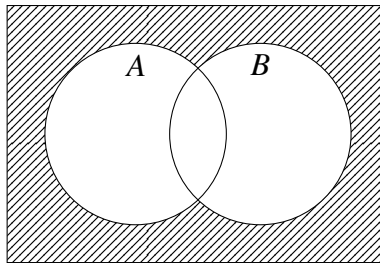
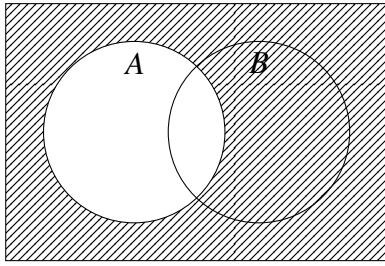
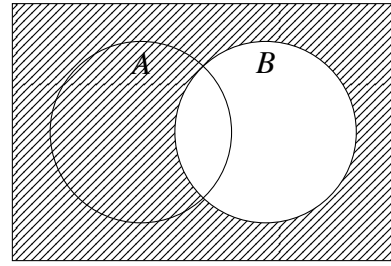


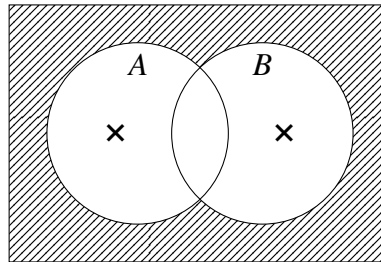
FIGURE 4 – Le diagramme de p_4 .

Le diagramme de la conclusion étendue $p_5 \vee p_6$ est constitué des deux diagrammes suivants :

-
3. À condition de considérer l'axiome du choix, qui implique lui même le principe du tiers exclu
 4. *Ne pas être vide* et *Contenir un élément* sont des propriétés équivalentes grâce à l'axiome du choix.

FIGURE 5 – Le diagramme de p_5 .FIGURE 6 – Le diagramme de p_6 .

Ici, il est possible d'étendre le diagramme de p_4 en fixant les zones $A \cap \neg B$ et $\neg A \cap B$ comme non-vides, ce qui donne le diagramme D'' suivant :

FIGURE 7 – Le diagramme de p_4 étendu.

On pourra vérifier que ce diagramme est compatible avec le diagramme de p_4 , mais que ce diagramme est *en contradiction* avec les deux de la conclusion étendue. Il y a contradiction avec le diagramme de p_5 , car la zone $A \cap \neg B$ y est vide alors qu'elle est non-vide dans D'' . De même, il y a contradiction avec le diagramme de p_6 , parce que la zone $B \cap \neg A$ y est vide tandis qu'elle est non-vide dans D'' .

Le syllogisme est donc invalide, car on a trouvé une extension du diagramme de la prémisse en contradiction avec chaque diagramme de la conclusion étendue.

6 Formalisation des notions

Suite aux exemples des sections précédentes, on peut formaliser plusieurs notions. On dira que :

- un diagramme est *complet* s'il admet une contrainte de vacuité ou de non-vacuité sur chaque zone.
- deux diagrammes D et D' sont *contradictoire*s s'il existe une zone qui est non-vide dans D et vide dans D' , ou inversement.
- un diagramme D est une *extension* d'un diagramme D' s'il est obtenu en ajoutant des contraintes sur des zones non définies de D' .
- un syllogisme complexe est *invalide* s'il existe un diagramme D construit à partir des prémisses et une extension D_e de D telle que D_e est en contradiction avec chaque diagramme construit à partir de la conclusion étendue. Sinon, le syllogisme complexe sera dit *valide*.

Question 1

Expliquer pourquoi les propriétés suivantes sont valides :

1. Si des diagrammes D et D' sont contradictoires, alors D n'est pas compatible avec D' .
2. Si des diagrammes D et D' sont contradictoires, alors toute extension de D est en contradiction avec D' .
3. Un diagramme D est compatible avec un diagramme D' si et seulement si D est une extension de D' .

4. Le nombre d'extensions (respectivement extensions complètes) d'un diagramme de Venn est 3^k (respectivement 2^k), où k est le nombre de zones non définies dans le diagramme.
5. Si D est un diagramme complet, alors deux diagrammes D et D' ne sont pas contradictoires si et seulement si D est compatible avec D' .

7

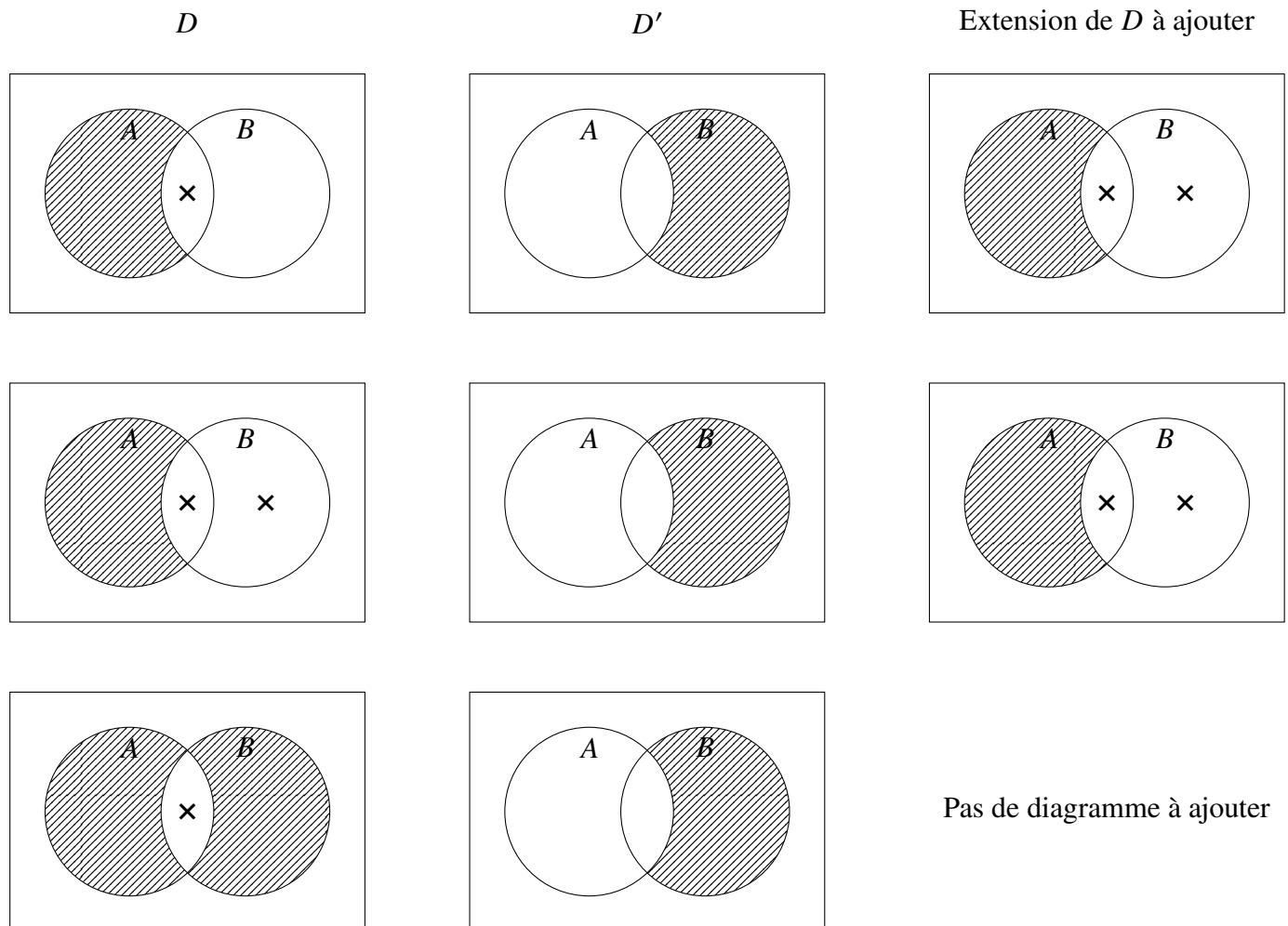
La validité des syllogismes : la négation entre en jeu

Au lieu de construire l'intégralité des extensions complètes d'un diagramme des prémisses afin de déterminer s'il en existe une en contradiction avec chaque diagramme de la conclusion étendue, on va plutôt essayer de procéder étape par étape, en commençant par considérer chaque diagramme de la conclusion séquentiellement.

En effet, si on considère un diagramme D des prémisses et un diagramme D' de la conclusion, on peut décider de l'existence d'une extension D_e de D qui est en contradiction avec D' en construisant la liste de diagrammes suivante : pour chaque zone z ayant une contrainte définie comme vide (respectivement non-vide) de D' ,

1. si z est non définie dans D , alors on ajoute à la liste le diagramme D étendu en fixant la zone z comme non-vide (respectivement vide),
2. si z est non-vide (respectivement vide) dans D , alors on ajoute à la liste le diagramme D inchangé,
3. si z est vide (respectivement non-vide) dans D , alors on laisse la liste inchangée.

Par exemple, en considérant la zone $(\neg A \cap B)$, les trois cas précédents sont illustrés par les diagrammes suivants :



On pourra alors remarquer avec cette construction que :

- si D est compatible avec D' , alors la liste construite est vide,
- si la liste construite est non-vide, alors chaque diagramme de cette liste est une extension de D en contradiction avec D' .

Cette construction peut se réaliser très simplement en considérant la *négation* d'un diagramme D . Un diagramme peut être vu comme une conjonction de contraintes de (non) vacuité. Ainsi, la négation d'un diagramme D peut être vue comme une disjonction de telles contraintes *inversées*.

Plus précisément, la *négation* d'un diagramme D est la liste des diagrammes obtenus en considérant chaque zone z de D définissant une contrainte c , et en construisant un diagramme D_z contenant une seule contrainte, la contrainte inverse de c pour la zone z .

Par exemple, la négation du diagramme de p_5 , représenté à la Figure 5, serait la liste contenant les deux diagrammes suivants :

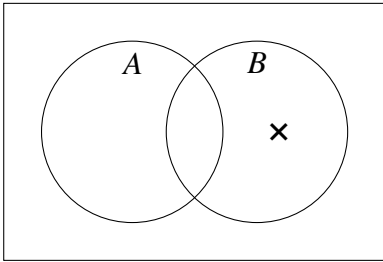


FIGURE 8 – Le premier diagramme de la négation de celui de p_5 .

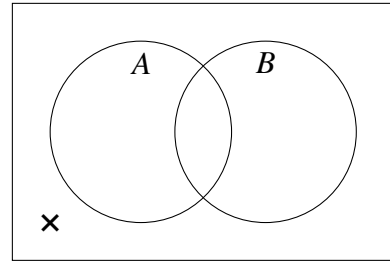


FIGURE 9 – Le second diagramme de la négation de celui de p_5 .

D'une façon équivalente, la négation du diagramme de p_6 , représenté à la Figure 6, serait la liste contenant les deux diagrammes suivants :

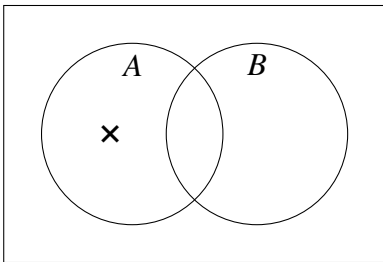


FIGURE 10 – Le premier diagramme de la négation de celui de p_6 .

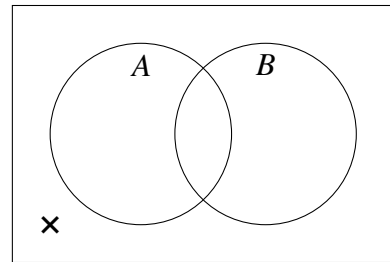


FIGURE 11 – Le second diagramme de la négation de celui de p_6 .

La construction précédente revient alors à réaliser la conjonction de deux listes, la première étant la liste contenant uniquement le diagramme D , et la seconde étant la négation du diagramme D' . En répétant cette étape pour chaque diagramme de la conclusion étendue, on obtient une liste de diagrammes qui sont des extensions de D en contradiction avec chaque diagramme de la conclusion étendue. Après avoir fait cela pour chaque diagramme D des prémisses, on obtient une liste de diagrammes qui est :

- soit vide, et alors le syllogisme est valide,
- soit non-vide, et alors cette liste contient des extensions des diagrammes des prémisses qui sont en contradiction avec chaque diagramme de la conclusion étendue.

Afin de faciliter l'écriture, on étendra la notation utilisée précédemment pour les syllogismes aux listes de diagrammes. Ainsi, on notera par

$$[d_1, \dots, d_n] \stackrel{?}{\vdash} [d'_1, \dots, d'_m]$$

le questionnement de la validité du syllogisme complexe dont les prémisses ont pour diagrammes $d_1, \dots, d_n$ et dont la conclusion a pour diagrammes $d'_1, \dots, d'_m$. À l'aide de cette notation, l'opération précédente s'exprime récursivement comme suit :

$$\begin{aligned} [d_1, \dots, d_n] \vdash^? [d'_1, \dots, d'_m] &\Leftrightarrow [d_1, \dots, d_n] \wedge \neg d'_1 \vdash^? [d'_2, \dots, d'_m] \\ [d_1, \dots, d_n] \vdash^? [d'_1] &\Leftrightarrow [d_1, \dots, d_n] \wedge \neg d'_1 \vdash^? [] \end{aligned}$$

Par exemple, pour le syllogisme $p_4 \vdash^? p_5 \vee p_6$, il suffit tout d'abord de réaliser la conjonction de la liste contenant uniquement le diagramme de p_4 avec la négation du diagramme de p_5 (la liste des deux diagrammes présentés précédemment), produisant le diagramme suivant :

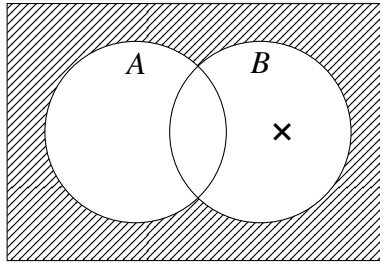


FIGURE 12 – Le diagramme obtenu après la première conjonction.

Ce diagramme est une extension du diagramme de p_4 en contradiction avec celui de p_5 . On répète ensuite l'opération en réalisant la conjonction de ce diagramme avec la négation du diagramme de p_6 (la liste des deux diagrammes présentés précédemment), produisant le diagramme suivant :

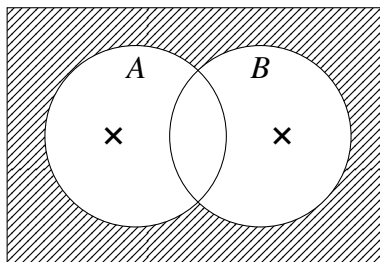


FIGURE 13 – Le diagramme obtenu après la seconde conjonction.

Ce diagramme est une extension du diagramme de p_4 en contradiction avec ceux de la conclusion étendue $p_5 \vee p_6$. Ainsi, le syllogisme $p_4 \vdash^? p_5 \vee p_6$ est invalide, comme attendu.

D'une façon encore plus rapide, en étendant la négation sur une liste de diagrammes comme suit,

$$\neg[d_1, \dots, d_n] = \neg d_1 \wedge \dots \wedge \neg d_n,$$

on peut montrer que

$$[d_1, \dots, d_n] \vdash^? [d'_1, \dots, d'_m] \Leftrightarrow [d_1, \dots, d_n] \wedge \neg[d'_1, \dots, d'_m] \vdash^? [].$$

Question 2



Expliquer pourquoi les propriétés suivantes sont valides :

1. La conjonction d'une liste de diagrammes ℓ avec la négation d'un diagramme d est vide si et seulement si tout diagramme de ℓ est compatible avec d .
2. La conjonction d'une liste de diagrammes ℓ avec la négation d'une liste de diagrammes ℓ' ne contient que des extensions de diagrammes de ℓ en contradiction avec d .

8 Idée générale de l'algorithme utilisé

Pour réaliser et tester empiriquement cela en OCaml, il faudrait donc :

1. calculer la conjonction de deux listes de diagrammes,
2. calculer la disjonction de deux listes de diagrammes,
3. calculer la négation d'un diagramme,
4. calculer la négation d'une liste de diagrammes,
5. calculer les diagrammes d'une combinaison Booléenne de formules de syllogismes,
6. tester si deux diagrammes sont contradictoires,
7. tester naïvement l'invalidité d'un syllogisme complexe, c'est-à-dire déterminer s'il existe une extension complète d'un diagramme des prémisses en contradiction avec chaque diagramme de la conclusion, et renvoyer des témoins le cas échéant,
8. tester la validité d'un syllogisme *via* les conjonctions et négations, c'est-à-dire déterminer si la conjonction des diagrammes des prémisses avec la négation des diagrammes de la conclusion est la liste vide, et renvoyer des témoins le cas échéant.

Les sections suivantes présentent les différents éléments à réaliser dans les fichiers fournis disponibles sur universitice.

9 Implantation : structure des ressources de départ

La structure du projet et l'utilisation de dune sont imposées. La programmation impérative n'est pas acceptée : ainsi, pas de boucle ni de variable mutable. La structure de départ, à télécharger sur universitice, est la suivante :

- un dossier Proposition contenant le fichier Formule.ml, à remplir avec les définitions du TP1 que vous avez rédigées en TP ;
- un dossier Predicats contenant :
 - le fichier Formule_Syllogisme.ml, à remplir avec les définitions du TP4 et contenant une définition de type, boolCombSyllogismes, pour les syllogismes complexes ;
 - le fichier DiagVenn.ml, à remplir avec les définitions du TP4 que vous avez rédigées en TP, et à compléter par de nouvelles fonctions prédéfinies ;
 - le fichier ValiditeNaive.ml à compléter, pour tester naïvement la validité des syllogismes complexes et chercher des témoins d'invalidité le cas échéant ;
 - le fichier ValiditeViaNegation.ml à compléter, pour tester la validité des syllogismes complexes *via* les conjonctions et négations, et chercher des témoins d'invalidité le cas échéant.

Pour ouvrir ces modules automatiquement lors du chargement de dune utop et effectuer des tests, vous pourrez ajouter un fichier .ocamlinit.

9.1 Le fichier Formule.ml

Vous devez ajouter dans ce fichier les fonctions que vous aviez écrites en TP1 pour manipuler les formules propositionnelles. Plus particulièrement, vous devez construire des tables de vérité afin de pouvoir construire les diagrammes de Venn.

Cette étape est fondamentale : sans celle-ci, vous ne pourrez pas être évalués sur le projet, qui sera nécessairement instable.

9.2 Le fichier `Formule_Syllogisme.ml`

Vous devez compléter ce fichier avec les éléments du TP4. À ceux-ci s'ajoute une nouvelle définition de type, que vous ne devez pas modifier, permettant de représenter les prémisses et conclusions des syllogismes complexes :

```
(** Type des combinaisons booléennes pour les syllogismes *)
type boolCombSyllogismes =
| Vrai
| Faux
| Base of formule_syllogisme
| Et of boolCombSyllogismes * boolCombSyllogismes
| Ou of boolCombSyllogismes * boolCombSyllogismes
| Non of boolCombSyllogismes
```

Par exemple, il est possible de définir des formules comme suit :

```
(* b1 : pour tout x, a(x) ou b(x) *)
let b1 = Base (PourTout (Ou (Atome "a", Atome "b"))))

(* b2 : (pour tout x, a(x)) ou (pour tout x, b(x)) *)
let b2 = Ou (Base (PourTout (Atome "a"))), Base (PourTout (Atome "b"))))
```

9.3 Le fichier `DiagVenn.ml`

Dans ce fichier, vous devez ajouter le code réalisé pour le TP4. En plus de celui-ci, vous devez implanter les fonctions suivantes, sans modifier les signatures fournies :

- `negate_diag`, qui calcule la négation d'un diagramme de Venn ;
- `negate_diag_list`, qui calcule la négation d'une liste de diagrammes de Venn ;
- `conj_diag_list`, qui calcule la conjonction de deux listes de diagrammes de Venn ;
- `disj_of_diag_list`, qui calcule la disjonction de deux listes de diagrammes de Venn ;
- `diags_of_bool_comb`, qui calcule les diagrammes de Venn associés à une combinaison Booléenne de formules de syllogismes.

9.4 Le fichier `ValiditeNaive.ml`

Dans ce fichier, vous devez implanter les fonctions suivantes, sans changer les signatures fournies :

- `complete_diags`, qui calcule les extensions complètes d'un diagramme de Venn pour un alphabet de référence donné. Celui-ci est nécessaire, puisqu'un diagramme défini par une contrainte vide sur une zone A pourrait être complété en considérant les zones définies par une liste d'atomes $[A; B; C]$;
- `is_contradiction`, qui teste si deux diagrammes de Venn sont contradictoires ;
- `est_valid_premiss_conc`, qui teste naïvement la validité d'un syllogisme complexe, défini par deux valeurs de type `boolCombSyllogismes`, une pour les prémisses et une pour la conclusion ;
- `temoins_invalidite_premisses_conc`, qui calcule naïvement une liste de témoins d'invalidité pour un syllogisme complexe, défini par deux valeurs de type `boolCombSyllogismes`, une pour les prémisses et une pour la conclusion. Ces témoins sont les diagrammes de Venn des prémisses qui peuvent être complétés en des diagrammes en contradiction avec chaque diagramme de la conclusion.

9.5 Le fichier `ValiditeViaNegation.ml`

Dans ce fichier, vous devez implanter les fonctions suivantes, sans changer les signatures fournies :

- `est_valid_premiss_conc`, qui teste la validité d'un syllogisme complexe, défini par deux valeurs de type `boolCombSyllogismes`, une pour les prémisses et une pour la conclusion. La validité est testée par la vacuité de la liste obtenue en réalisant la conjonction des diagrammes des prémisses avec la négation des diagrammes de la conclusion ;
- `temoins_invalidite_premisses_conc`, qui calcule une liste de témoins d'invalidité pour un syllogisme complexe, défini par deux valeurs de type `boolCombSyllogismes`, une pour les prémisses et une pour la conclusion. Ces témoins les diagrammes de la conjonction des diagrammes des prémisses avec la négation de la conclusion, qui sont ainsi des extensions des diagrammes des prémisses en contradiction avec chaque diagramme de la conclusion.

10 Barème indicatif

1. Code (10 points) :

(a) Fichier `DiagVenn.ml` (3 points) :

- négation, conjonction et disjonction de listes de diagrammes (1.5 point)
- construction des diagrammes de Venn d'une combinaison booléenne de formules pour syllogismes (1.5 points)

(b) Module `ValiditeNaive.ml` (5 points) :

- calcul des extensions complètes d'un diagramme (1 point)
- test de contradiction de deux diagrammes (1 point)
- validité et recherche de témoins d'invalidité d'un syllogisme complexe par la méthode naïve (3 points)

(c) Module `ValiditeViaNegation.ml` (2 points) :

- validité et recherche de témoins d'invalidité d'un syllogisme complexe par la méthode *via* négation (2 points)

2. Rapport et soutenance (10 points) :

- Compréhension des algorithmes, y compris ceux du TP
- Réponses aux questions de l'énoncé du projet
- Autres éléments attendus

Il s'agit d'un barème indicatif "de départ". À celui-ci s'ajouteront différentes pénalités, notamment si :

- votre code rencontre des erreurs de compilation ⁵ ;
- votre code émet des avertissements à la compilation ⁶ ;
- des contre-exemples sont détectés ⁷,

ainsi que des bonus, notamment si :

- votre rapport est écrit en $\text{\LaTeX}$,
- vous codez d'une façon propre, lisible, élégante et efficace.

NB : Les algorithmes mis en oeuvre dans ce projet doivent être connus et seront testés lors de la soutenance orale. En cas de méconnaissance manifeste des algorithmes, la note pourra être réduite de manière significative, en distinguant les notes entre les membres d'un même binôme le cas échéant.

5. note pouvant être réduite à 0.

6. note pouvant être réduite sous la moyenne.

7. note pouvant être réduite à 0 pour la fonction concernée.